DINOSECURITIES · INTERNAL TECH ROADMAP

Platform Enhancements.

Eight proposed tech updates for the DinoSecurities web app — scoped, designed, and cost-estimated. Each adds a meaningful trust signal or growth mechanism without requiring audit-level changes to the on-chain programs.

VERSION	DATE	AUDIENCE
Draft 0.1	April 15, 2026	Internal · Dev + Exec

Introduction & **Scope.**

DinoSecurities is live on Solana mainnet-beta: three Anchor programs deployed, atomic Delivery-vs-Payment settlement working end-to-end, a frontend at www.dinosecurities.com, and a backend on DigitalOcean with Didit KYC wired in. The soft launch is operational.

The items in this document are **optional enhancements** — none of them are blocking the current product. They're proposed because each one either (a) makes a real regulatory or trust claim *demonstrable* rather than asserted, (b) opens a distribution channel we don't currently have, or (c) unlocks a use case no competitor in regulated-securities tokenization is doing on Solana today.

Each section answers four questions:

- **What is it?** One paragraph you could read to a non-technical founder or investor.
- **Why does it matter?** The trust signal, compliance argument, or growth mechanism it creates.
- **How does a dev build it?** Concrete implementation plan — files touched, APIs called, algorithms.
- **What does it cost?** Effort estimate + external dependencies + risk of breakage.

READING THE EFFORT BADGES

EASY — 2–4 focused hours. One dev, one PR.

MEDIUM — 4–8 hours. May touch backend + frontend together.

HARDER — 1–2 days. New dependencies, cross-cutting changes, or novel design work.

TABLE OF CONTENTS

01 Click-to-Verify Landing Page Stats — Easy · ~3 h

02 Client-Side Ricardian Document Verifier — Easy · ~2 h

03 Pre-Trade Compliance Simulator — Medium · ~4 h

04 Live Mainnet Settlement Ticker — Medium · ~3 h

05	Holder Geography Heatmap per Series	Easy · ~3 h
06	Trade-Confirmation (Rule 10b-10) PDF Receipts	Medium · ~4 h
07	Embeddable Issuer Widget	Medium · ~3 h
08	Soulbound Investor Passport NFT	Harder · ~5 h

Click-to-Verify Landing Page Stats

EFFORT	DIFFICULTY	TOUCHES	DEPENDENCIES
~3 hours	Easy	Frontend only	None

WHAT IT IS

Every performance and cost claim on the landing page — "400ms finality", "<$0.01 settlement cost", "sub-second DvP", "atomic, zero counterparty risk" — becomes a clickable link. Clicking opens a live Solana Explorer tab pointed at a *real, recent* mainnet settlement transaction that provably demonstrates the claim.

Competitors say "fast settlement" — we link to the tx.

WHY IT MATTERS

Tokenized-securities platforms are swimming in unverifiable marketing numbers. The instinctive reaction of any sophisticated issuer, auditor, or regulator reading a claim like "99.9% uptime" or "sub-second finality" is *prove it*. Click-to-verify makes the proof inline, no support ticket required. It also disincentivizes anyone on our team from ever putting a number on the site that isn't backed by real on-chain data — the click would return a 404-shaped hole.

HOW A DEV BUILDS IT

1. **Create a backend endpoint** `GET /trpc/analytics.recentSettlements` that returns the 5 most recent `SettlementExecuted` events from the `webhook_events` table, shaped as `{ signature, blockTime, slot, dvpTimeMs }`.
2. **On page load in** `PlatformSection.tsx`, fetch one row; memoize for 60 seconds via TanStack Query.
3. **Replace the static** `"400ms finality"` **text** with a `<Link>` that renders `${confirmationTime}ms finality` (computed from the real tx's blockTime vs submission time stored in `settlement_orders`) and `href` to `https://explorer.solana.com/tx/${signature}`.

4. **Do the same for fee claims** (`fee = meta.fee / 1e9` from the cached tx receipt) and atomicity (signature links to the tx itself, which shows both legs in one instruction).

Code sketch for the numbers:

```
// server/src/routers/analytics.ts
recentSettlements: publicProcedure.query(async ({ ctx }) => {
  const rows = await ctx.db
    .select()
    .from(settlementOrders)
    .where(eq(settlementOrders.status, 'settled'))
    .orderBy(desc(settlementOrders.settledAt))
    .limit(5);
  return rows.map(r => ({
    signature: r.settlementTxSignature,
    finalityMs: r.finalityMs ?? 400,
    feeSol: r.feeSol ?? 0.0001,
  }));
})
```

WHAT COULD GO WRONG

If no settlements have happened in N days, the endpoint returns nothing and the landing page falls back to static numbers with a small "avg over last 7 days" note. Cache-bust the query on deploy so stale claims don't linger after a bug fix.

COST SUMMARY

LINE ITEM	HOURS
Backend router additions + query	0.5
Frontend wire-up in PlatformSection + FeaturesGrid	1.5
Styling the click-affordance (tiny arrow icon)	0.5
QA + fallback path	0.5
Total	**~3 hours**

UPDATE 02 / EIGHT

Client-Side Ricardian Document Verifier

EFFORT	DIFFICULTY	TOUCHES	DEPENDENCIES
~2 hours	Easy	Frontend only	None

WHAT IT IS

On every SecurityDetail page, the browser fetches the offering document from Arweave, computes its SHA-256 hash locally (in JavaScript, zero trust in any server), and displays a big green "✅ Document matches on-chain hash" badge — or red "⚠ Document has been altered since issuance" if it doesn't.

The on-chain `SecuritySeries` PDA already stores the `doc_hash: [u8; 32]` field. The client just has to recompute it.

WHY IT MATTERS

The whole *point* of a Ricardian contract is that the legal document is cryptographically bound to the token. Today that binding exists on-chain, but no user ever sees it verified. Every investor has to take on faith that the PDF they're reading is the one the hash points at.

Showing the verification happen **in the browser** — not on our server — is the strongest possible trust signal. We could be compromised and the verification would still be honest: either the Arweave bytes match the on-chain hash or they don't, and anyone can re-run the check with browser devtools.

This is the kind of thing a Big-Four audit firm or a securities lawyer would include in a due-diligence checkbox: "Can the investor verify the legal document has not been substituted?" Today that question is awkward. After this ships, the answer is "yes, live, in their browser."

HOW A DEV BUILDS IT

1. **In** `src/hooks/useDocVerification.ts`, create a hook that takes `{ docUri, expectedHash }`.

2. **Fetch the Arweave bytes:** normalize `docUri` (handle `ar://…` and `https://arweave.net/…` forms), call `fetch(url)`, read as ArrayBuffer.
3. **Hash in the browser** using SubtleCrypto (`window.crypto.subtle.digest('SHA-256', buffer)`). This is native, zero dependencies.
4. **Compare hex-encoded hash** against the `expectedHash` bytes read from the on-chain SecuritySeries PDA.
5. **Render a small status component** with three states: `verifying…` (spinner), `matches` (green check), `mismatch` (red warn).

```
// src/hooks/useDocVerification.ts
export function useDocVerification(docUri: string, expectedHex: string) {
  return useQuery({
    queryKey: ['doc-verify', docUri],
    queryFn: async () => {
      const url = docUri.startsWith('ar://')
        ? `https://arweave.net/${docUri.slice(5)}`
        : docUri;
      const buf = await (await fetch(url)).arrayBuffer();
      const hash = await crypto.subtle.digest('SHA-256', buf);
      const hex = [...new Uint8Array(hash)]
        .map(b => b.toString(16).padStart(2, '0')).join('');
      return { matches: hex === expectedHex.toLowerCase(), hex };
    },
    staleTime: 5 * 60_000,
  });
}
```

WHAT COULD GO WRONG

- **CORS on Arweave gateways** — most `arweave.net/*` responses send permissive CORS. If a mint used a gateway that blocks CORS, the fetch fails and we fall back to "unverified" (not "mismatch").
- **Large documents** — a 50MB PDF hashes in a second or two on desktop, much longer on mobile. Stream-hash if we care about that edge case.

COST SUMMARY

LINE ITEM	HOURS
Hook + SubtleCrypto implementation	0.75
Status component (3 states)	0.5
Wire into SecurityDetail.tsx	0.5
Edge cases (CORS, timeouts)	0.25
Total	**~2 hours**

UPDATE 03 / EIGHT

Pre-Trade Compliance Simulator

EFFORT	DIFFICULTY	TOUCHES	DEPENDENCIES
~4 hours	Medium	Frontend + Backend	Existing RPC

WHAT IT IS

A public page (no wallet connection required) where anyone can paste a wallet address and pick a security series, and immediately see a pass/fail table showing exactly what would happen if that wallet attempted to receive the token.

Output looks like:

SIMULATOR OUTPUT — EXAMPLE

Wallet: 4e4Xf6Zt...NyTM → **Series:** DINOMT (DINOMAIN)

✅ Whitelisted as a holder
✅ KYC verified — not expired (expires 2026-11-03)
✅ Not frozen
✅ Not revoked
✅ Accreditation: **accredited** (required for Reg D)
❌ Jurisdiction: US (blocked by Reg S)

Result: Transfer would be rejected — `RegSUsPersonBlocked`

WHY IT MATTERS

Every securities tokenization platform in existence claims "on-chain compliance." The difference between a claim and a moat is whether an outsider can independently verify it without signing a transaction. The simulator does exactly that:

- **For regulators and auditors** — they can run the same checks we do, see the source of truth is the chain itself, not our backend.
- **For prospective issuers evaluating us** — they type in their own test wallet and see the hook refuse to let a non-accredited investor receive an accredited-only token.

Proof the infrastructure works, not a demo video they have to trust.

- **For sophisticated investors** — they check their own eligibility before they touch the wallet popup.

There is no marketing copy that beats "run it yourself."

HOW A DEV BUILDS IT

The simulator is entirely read-only. We replicate the transfer-hook checks off-chain in TypeScript by fetching the relevant accounts and evaluating the same conditions:

1. **New tRPC route** `compliance.simulate({ wallet, mint })`.
2. **Load the accounts** via Anchor's `BorshAccountsCoder`:
   - `SecuritySeries` PDA at `["series", mint]`
   - `PlatformConfig` PDA at `["platform"]`
   - `HolderRecord` PDA at `["holder", mint, wallet]` (may not exist — that's result #1)
3. **Run the same checks** the hook runs, in the same order:

```
const series = await fetchSeries(mint);
const holder = await fetchHolder(mint, wallet); // may be null
const checks = [
  { name: 'whitelisted', pass: !!holder, detail: holder ? '' : 'No HolderRecord PDA
  { name: 'not revoked', pass: !holder?.isRevoked },
  { name: 'not frozen', pass: !holder?.isFrozen },
  { name: 'kyc not expired',
    pass: Number(holder?.kycExpiry ?? 0) > Date.now() / 1000 },
  { name: 'series not paused', pass: !series.paused },
];
if (series.transferRestriction === 'RegD') {
  checks.push({ name: 'accredited (Reg D)', pass: holder?.isAccredited === true });
}
if (series.transferRestriction === 'RegS') {
  checks.push({
    name: 'non-US jurisdiction (Reg S)',
    pass: Buffer.from(holder?.jurisdiction ?? []).toString() !== 'US',
  });
}
return { pass: checks.every(c => c.pass), checks };
```

4. **Frontend page** `src/pages/ComplianceSimulator.tsx` — two inputs (wallet + series dropdown), one button, a table showing each check with its ✅/❌ and any detail text.

WHAT COULD GO WRONG

- **Simulator drift** — if we update the hook's logic without updating the simulator, they diverge. Mitigation: add a CI test that walks a fixture set of (wallet, mint) cases and asserts the simulator and an on-chain dry-run agree. Easy to add later.
- **RPC rate limits** — a popular simulator page could hammer Helius. Cache each (wallet, mint) result for 60 seconds.

COST SUMMARY

LINE ITEM	HOURS
Backend: account fetch + check logic	1.25
Backend: tRPC route + caching	0.5
Frontend: page + form	1.0
Frontend: result table styling	0.75
Fixture testing	0.5
Total	**~4 hours**

Live Mainnet Settlement Ticker

EFFORT	DIFFICULTY	TOUCHES	DEPENDENCIES
~3 hours	Medium	Frontend + Helius	Existing webhook

WHAT IT IS

A Bloomberg-style ticker that scrolls horizontally across the bottom of the landing page hero, showing the last several real mainnet settlements as they happen:

`DINOMT × 10 @ $1.00 · 2.3s ago     APPLE-S × 500 @ $180.00 · 14s ago     FUND-ABC × 25 @ $1,000 · 38s ago`

New settlements appear at the left end in real time via a WebSocket push the moment the webhook handler persists them. The marquee wraps and repeats.

WHY IT MATTERS

Static landing pages feel like brochure sites. A moving, real-time feed of actual mainnet activity signals to every visitor that the platform is *alive* — not a pitch-deck concept with a Figma mockup behind it. Even one visible settlement per minute communicates more than a thousand words of copy.

For investor demos: instead of saying "we've done over a thousand settlements on mainnet," you point at the ticker and let them watch one happen. It's also a passive integrity check — if the ticker ever stops while we claim to be operational, we know before our users do.

HOW A DEV BUILDS IT

Two halves: a backend broadcast and a frontend subscriber.

Backend broadcast layer. Express-native `ws` package, one WebSocket endpoint at `/ws/settlements`. On every `SettlementExecuted` event persisted by the Helius webhook handler, also `broadcast(JSON.stringify(shape))` to all connected clients.

```
// server/src/ws.ts
import { WebSocketServer } from 'ws';
const wss = new WebSocketServer({ noServer: true });
export function broadcastSettlement(s: SettlementRow) {
  const msg = JSON.stringify({
    type: 'settlement',
    symbol: s.symbol, amount: s.tokenAmount, price: s.priceUsd,
    at: s.settledAt, signature: s.signature,
  });
  for (const ws of wss.clients) {
    if (ws.readyState === 1) ws.send(msg);
  }
}
// in index.ts
server.on('upgrade', (req, sock, head) => {
  if (req.url === '/ws/settlements') {
    wss.handleUpgrade(req, sock, head, ws => wss.emit('connection', ws, req));
  }
});
```

Frontend subscriber. `src/components/veliq/SettlementTicker.tsx`. On mount, connect `new WebSocket('wss://…/ws/settlements')`. Seed with last 10 via existing `analytics.recentSettlements` route. Push new items to the left of an array capped at 20. Render as a Framer-Motion horizontal marquee.

WHAT COULD GO WRONG

- **No settlements happen** — on a quiet day the ticker looks stale. Fallback: if last settlement > 1 hour old, show "Live feed · awaiting activity" in a muted style.
- **WebSocket reconnection** — basic exponential-backoff reconnect (1s, 2s, 4s... cap at 30s). Trivial, add from the start.
- **DigitalOcean WebSocket support** — DO App Platform supports WS out of the box, no special config.

COST SUMMARY

LINE ITEM	HOURS
Backend ws server + broadcast	1.0
Wire broadcast into webhook handler	0.25
Frontend component + marquee	1.25
Reconnect + fallback states	0.5
Total	**~3 hours**

UPDATE 05 / EIGHT

Holder Geography Heatmap per Series

EFFORT	DIFFICULTY	TOUCHES	DEPENDENCIES
~3 hours	Easy	Frontend only	d3-geo or react-simple-maps

WHAT IT IS

On each SecurityDetail page, a world map colors each country by how many whitelisted holders are registered from that jurisdiction. Hover over a country for `"United States — 42 holders"`. No names, no wallet addresses, no amounts — just counts per country code.

Data source: the existing `HolderRecord.jurisdiction` field (2 bytes, ISO-3166 alpha-2 codes) across all holders for a given mint.

WHY IT MATTERS

Issuers love geographic distribution charts for their own pitch decks to secondary investors and board reports. Today they'd have to export data and build the chart themselves. Building it into the platform means the issuer-portal page becomes a self-service demo-deck generator, which is a retention mechanism disguised as a feature.

Secondary benefit: compliance. An issuer running a Reg S series can visually confirm there are no US holders listed. Mistakes in jurisdiction fields become visually obvious.

HOW A DEV BUILDS IT

1. **Backend:** add `holders.byJurisdiction(mint)` returning `[{ iso2: 'US', count: 42 }, …]`. Simple `GROUP BY jurisdiction` against the `indexed_holders` table.
2. **Frontend:** install `react-simple-maps` (lightweight, SVG-based, no D3 knowledge needed) and pull a world GeoJSON.
3. **Color scale:** quantize counts into 5 buckets with d3-scale. Use the platform purple as the high end.
4. **Tooltip:** hook into the map's `onMouseEnter` per-country to show the count.

5. **Anonymization:** bucket `count < 5` into "<5 holders" to avoid de-anonymization in sparse countries.

```
// src/components/HolderGeoMap.tsx
import { ComposableMap, Geographies, Geography } from 'react-simple-maps';
import worldGeo from './world-countries.json';

export function HolderGeoMap({ data }: { data: Record<string, number> }) {
  const max = Math.max(...Object.values(data), 1);
  return (
    <ComposableMap projection="geoMercator">
      <Geographies geography={worldGeo}>
        {({ geographies }) =>
          geographies.map(geo => {
            const iso = geo.properties.ISO_A2;
            const count = data[iso] ?? 0;
            const opacity = count / max;
            return (
              <Geography
                key={geo.rsmKey} geography={geo}
                fill={count > 0 ? `rgba(139, 92, 246, ${0.15 + opacity * 0.85})` : '
                stroke="hsl(0,0%,20%)"
              />
            );
          })
        }
      </Geographies>
    </ComposableMap>
  );
}
```

WHAT COULD GO WRONG

- **Empty data** — if a series has no whitelisted holders yet, show a placeholder state instead of an empty gray map.
- **De-anonymization risk** — per above, always bucket small counts. A holder in Luxembourg with count=1 is effectively named.

COST SUMMARY

LINE ITEM	HOURS
Backend aggregate route	0.5
Install map package + import GeoJSON	0.5
Map component + color scaling	1.0
Tooltip + legend	0.5
Wire into SecurityDetail + IssuerPortal	0.5
Total	**~3 hours**

UPDATE 06 / EIGHT

Trade-Confirmation (Rule 10b-10) PDF Receipts

EFFORT	DIFFICULTY	TOUCHES	DEPENDENCIES
~4 hours	Medium	Backend (pdfkit) + email	pdfkit, Resend or SES

WHAT IT IS

Every successful DvP settlement automatically generates a formal trade-confirmation PDF — the kind of document a broker-dealer would mail after a securities transaction. It includes: both counterparties' identifiers, the CUSIP/ISIN or on-chain mint, quantity, price, total consideration, trade date, settlement date (same — it's atomic), the Solana transaction signature, and a cryptographic fingerprint of the confirmation itself.

If the buyer has provided an email in Settings, the PDF is auto-mailed. A signed copy is also downloadable from their Portfolio → Activity history for 7 years (the SEC retention period).

WHY IT MATTERS

U.S. securities regulation **requires** broker-dealers to send a confirmation after every trade — SEC Rule 10b-10. The platform isn't a broker-dealer, but any serious issuer operating on DinoSecurities needs to produce 10b-10-equivalent documentation for their holders' records or risk compliance findings during audit.

Competing tokenization platforms hand-wave this as "the issuer's problem." Producing the receipt automatically is a concrete, differentiated service that speaks directly to what securities counsel at an issuer will ask for in their onboarding call: *"How do your investors document their trades for their accountant?"*

Secondary win: the receipt itself serves as a marketing artifact. It's branded, professionally formatted, and includes a "Verify on Solana" link — every trade is a trust-building PDF sitting in a holder's inbox.

HOW A DEV BUILDS IT

1. **Install** `pdfkit` in `server/`. It's zero-dependency, streaming, and produces small clean PDFs.
2. **Create** `server/src/services/trade-receipt.ts` with a function `buildReceipt(settlement)` that returns a `Buffer`.
3. **Template the document.** Title block, the two parties, a trade-detail table, a regulatory footer, an on-chain verification section (link + truncated sig + QR code linking to Explorer).
4. **Generate on the** `SettlementExecuted` **webhook path** — after persisting the settlement row, enqueue a receipt job.
5. **Store** under `/receipts/{signature}.pdf` in DigitalOcean Spaces (or local disk with rclone backup) — cheap, predictable.
6. **Mail** via Resend (simplest) or SES. Include the PDF as an attachment. If no email on file, skip mail but still store.
7. **Expose** `trpc.settlements.receiptUrl(signature)` so the frontend Portfolio → Activity tab can link each row.

Template sketch:

```
function buildReceipt(s: SettlementRow): Buffer {
  const doc = new PDFDocument({ size: 'LETTER', margin: 56 });
  const chunks: Buffer[] = [];
  doc.on('data', c => chunks.push(c));

  doc.fontSize(10).fillColor('#6b6b7a').text('TRADE CONFIRMATION', { characterSpacin
  doc.moveDown(0.2);
  doc.fontSize(22).fillColor('#0a0a0f').text(`${s.symbol} × ${s.tokenAmount}`);
  doc.moveDown(0.5);
  addField(doc, 'Confirmation #', s.signature.slice(0, 12).toUpperCase());
  addField(doc, 'Trade date', s.settledAt.toISOString());
  addField(doc, 'Settlement date', s.settledAt.toISOString()); // same — atomic
  addField(doc, 'Buyer', truncate(s.buyer));
  addField(doc, 'Seller', truncate(s.seller));
  addField(doc, 'Mint', truncate(s.mint));
  addField(doc, 'Payment mint', 'USDC — EPjFWdd5…wyTDt1v');
  addField(doc, 'Consideration', `${s.paymentAmount / 1e6} USDC`);
  addField(doc, 'Unit price', `$${(s.paymentAmount / s.tokenAmount / 1e6).toFixed(4)
  // verification block
  doc.moveDown(1).fontSize(8).fillColor('#6b6b7a')
     .text(`Verify on Solana: https://explorer.solana.com/tx/${s.signature}`);
  doc.end();
  return Buffer.concat(chunks);
}
```

WHAT COULD GO WRONG

- **Email deliverability** — emailing PDFs can hit spam. Warm the sending domain via Resend's SPF/DKIM setup instructions before mailing real volumes.
- **Storage cost** — PDFs are ~20 KB each. Even at 100k/year, that's 2 GB/year. Nothing.
- **Regulatory scope creep** — the PDF should explicitly note "This confirmation is not a broker-dealer trade confirmation under Rule 10b-10. DinoSecurities is not a registered broker-dealer." to avoid implying things that aren't true.

COST SUMMARY

LINE ITEM	HOURS
`pdfkit` template + layout	1.5
Webhook integration + queue	0.75
DO Spaces storage setup	0.5
Resend integration + email template	0.75
Frontend link on Portfolio Activity	0.5
Total	**~4 hours**

UPDATE 07 / EIGHT

Embeddable Issuer Widget

EFFORT	DIFFICULTY	TOUCHES	DEPENDENCIES
~3 hours	Medium	Frontend (new route) + iframe bridge	None

WHAT IT IS

A `<script>` or `<iframe>` snippet that issuers paste on their corporate website. Rendered inside any page, it displays their series' live stats (supply, holder count, recent price, recent volume) plus an "Invest" button that opens the DinoSecurities flow in a popup or new tab.

Issuer copy-pastes something like:

```
<iframe
  src="https://www.dinosecurities.com/embed/DINOMT"
  width="380" height="520"
  style="border:0; border-radius: 8px;"
  loading="lazy"
  title="DINOMT — DinoSecurities">
</iframe>
```

And their shareholders see a live widget embedded in their own site.

WHY IT MATTERS

Distribution. Every issuer's corporate website is already trafficked by the exact people who might want to hold their securities. A widget lets them expose the investment flow directly there, without demanding the visitor learn a new platform or leave their site.

It also reframes DinoSecurities as *infrastructure* — the way Stripe Checkout, Substack embeds, or Calendly links all work. Issuers aren't sending users to us; they're using us on their own turf. That changes the pitch from "list with DinoSecurities" to "plug DinoSecurities into your site in five minutes."

HOW A DEV BUILDS IT

1. **New route** `src/pages/Embed.tsx` registered at `/embed/:symbol` (not under the `/app` prefix so it skips the authed-shell layout).
2. **Special layout** — no sidebar, no navbar, smaller fonts, single-column. Borders, padding, colors all controllable via URL query params (`?theme=light`, `?accent=%23ff6600`) for issuers with brand requirements.
3. **Data fetching** — use the same `securities.getByMint` and `holders.stats` tRPC routes. TanStack Query with a short stale time.
4. **Invest CTA** — the button opens `window.top.open('https://www.dinosecurities.com/app/marketplace/DINOMT?buy=1', '_blank')` — always a new tab, never trying to navigate the parent window (to sidestep cross-origin restrictions and to feel safer to visitors).
5. **Security headers** — set `Content-Security-Policy: frame-ancestors *` on the `/embed/*` routes so third-party sites are allowed to embed. This is a per-route Vercel config.
6. **Analytics** — capture the referring domain in a request header so we can report to issuers "this widget was rendered 1,200 times on acme-securities.com last week."

WHAT COULD GO WRONG

- **SEO + noindex** — `/embed/*` should return `<meta name="robots" content="noindex">` so Google doesn't crawl it as canonical.
- **Third-party cookie / local-storage restrictions in iframes** — Safari especially is aggressive. We shouldn't depend on any client state in the widget; each render is stateless.
- **Price manipulation** — if the widget shows a price, make sure it comes from the same source as the main marketplace, not a gameable one. For soft launch, use "last traded price" from on-chain settlements only.

COST SUMMARY

LINE ITEM	HOURS
`/embed/:symbol` route + layout	1.0
Theme/accent query-param handling	0.5
CSP + robots meta	0.25
Invest CTA + analytics	0.75
Documentation page ("How to embed")	0.5
Total	**~3 hours**

UPDATE 08 / EIGHT

Soulbound Investor Passport NFT

EFFORT	DIFFICULTY	TOUCHES	DEPENDENCIES
~5 hours	Harder	New program + frontend	Token-2022 NonTransferable

WHAT IT IS

After a user passes KYC with Didit, they can mint a non-transferable (soulbound) NFT to their wallet that cryptographically attests "this wallet was verified at accredited investor level, US jurisdiction, KYC expires 2027-Q3 — signed by DinoSecurities KYC oracle." Other Solana applications can read this NFT and decide whether to let the user participate, without having to run their own KYC flow.

The NFT isn't *authority* to do anything on DinoSecurities — we already check the HolderRecord PDA directly — it's a *portable credential* the user carries to the rest of the Solana ecosystem.

WHY IT MATTERS

Every RWA and regulated-DeFi protocol currently re-implements KYC. Each duplicates the same $10–30 per-user spend with Persona/Jumio/Didit. If DinoSecurities mints soulbound passports, we become one of the few *sources* of KYC attestations on Solana — and every downstream protocol that reads our passports becomes a distribution channel for our verification flow.

This is the same playbook as Civic, Worldcoin, or Sumsub, but targeted specifically at Solana securities-adjacent DeFi where our jurisdictional and accreditation signals actually matter. First mover advantage in a growing category.

Secondarily, for users: one KYC flow, many dApps. They see DinoSecurities as the thing that *let them into* Jupiter's accredited-only perps pool, lend on MarginFi's institutional vault, or buy on Magic Eden's RWA tier. We become infrastructure they associate with access.

HOW A DEV BUILDS IT

This is the only update in this doc that requires a new on-chain mint (per user passport) and a new UI flow. It does **not** require modifying existing programs.

1. **On-chain setup, per user:**
   - Create a Token-2022 mint with extensions `NonTransferable`, `MetadataPointer` (→ self), `PermanentDelegate` (→ KYC oracle, for revocation).
   - Mint exactly 1 token to the user's wallet.
   - Initialize `TokenMetadata` inline with:
     - Name: `DinoSecurities Investor Passport`
     - Symbol: `DINO-PASS`
     - URI: Arweave JSON with attributes `{ jurisdiction, accreditation, kyc_expires_at, oracle_signature }`
2. **Backend:** extend the Didit webhook handler — after a successful verification, in addition to creating the HolderRecord, enqueue a passport-mint job that signs the attributes with the KYC oracle keypair and returns the mint + metadata URI.
3. **Frontend:** after verified state is reached in Settings, surface a "Mint Investor Passport" button. One-tx flow, ~0.003 SOL rent.
4. **Public read API:** publish a small tRPC route `passport.verify(mint)` that returns `{ valid, attributes, revoked }` so third-party dApps can verify a passport without implementing Solana account decoding themselves.
5. **Revocation:** if a user's KYC expires or is revoked, the oracle uses the PermanentDelegate authority to burn the passport. Downstream dApps reading the passport see it gone.
6. **Documentation:** a short integration guide third parties can follow to consume passports: "Check if wallet X holds a DINO-PASS mint → read its metadata → respect the expiry."

WHAT COULD GO WRONG

- **Regulatory positioning** — saying "this wallet is an accredited investor" publicly on-chain is a signal regulators could treat as a representation. Legal should review the exact wording in the metadata and the terms of use users agree to at passport-mint time. Use careful language: "self-attested jurisdiction as confirmed via third-party KYC provider Didit on DATE" rather than "is an accredited investor."

- **Privacy tradeoff** — a soulbound passport tied to a wallet makes every transaction from that wallet linkable to a verified jurisdiction. Users should be able to opt out of minting. Frame it as optional.
- **Revocation latency** — if Didit revokes a user's verification and our oracle is delayed burning the passport, third parties could be operating on stale data for minutes to hours. Document the TTL conservatively.

COST SUMMARY

LINE ITEM	HOURS
Mint-creation helper (Token-2022 + extensions)	1.25
Oracle-signing + metadata JSON upload to Arweave	1.0
Frontend Settings CTA + mint tx	1.0
`passport.verify` public tRPC route	0.75
Integration guide (docs page)	0.75
Legal copy review pass	0.25
Total	**~5 hours**

Suggested **Rollout.**

If we ship all eight, there's a natural dependency order that minimizes thrash and maximizes the compounding benefit to the landing page, which is the highest-leverage surface.

PHASE A — TRUST SIGNALS (FIRST WEEK)

Do these three first. They're the lowest effort and the biggest impact per hour spent:

- **02 — Ricardian Verifier.** Ships a visible green check on every SecurityDetail page. 2 hours.
- **01 — Click-to-Verify Stats.** Turns every landing-page number into a live proof. 3 hours.
- **03 — Compliance Simulator.** Public, no-wallet page anyone can run. 4 hours.

Combined: ~9 hours, roughly two focused days. After this shipping round, the entire marketing message changes from "trust us" to "check it yourself."

PHASE B — DEMONSTRATION & RETENTION (SECOND WEEK)

- **04 — Live Settlement Ticker.** Landing page feels alive. 3 hours.
- **05 — Holder Geography Heatmap.** Issuer-portal stickiness. 3 hours.

Combined: ~6 hours, one day. These are not strictly necessary but pay back quickly in how the platform *feels* to new visitors and existing issuers.

PHASE C — DISTRIBUTION & COMPLIANCE (THIRD WEEK)

- **06 — PDF Trade Receipts.** The thing real issuers will ask for. 4 hours.
- **07 — Embeddable Widget.** Opens the distribution channel. 3 hours.

Combined: ~7 hours. These start to make DinoSecurities usable for an issuer who actually plans to run a real offering under proper compliance processes.

PHASE D — ECOSYSTEM (OPTIONAL, FOURTH WEEK)

- **08 — Soulbound Investor Passport.** The moat play. 5 hours.

This one isn't essential for the current product to function. It's essential for positioning DinoSecurities as *infrastructure the rest of the Solana RWA ecosystem builds on top of.* If the business strategy is "be a standalone tokenization platform," you can skip it indefinitely. If the strategy is "become the KYC layer for Solana securities," ship it.

TOTAL EFFORT IF ALL EIGHT SHIP

PHASE A	PHASE B	PHASE C	PHASE D
9 hours	**6 hours**	**7 hours**	**5 hours**

Grand total: **~27 focused hours** of engineering work, plus testing and polish. For one developer, that's ~3½ working days if done back-to-back, or ~3 weeks at a pace of an item every couple of days.

NOT IN SCOPE

This document does **not** cover:

- The Governance page rewrite (separate spec — requires `dino_governance` frontend wiring, proposal creation flows, and is significantly more involved than any item above).
- The on-chain lockup / holding-period enforcement (deferred, requires `HolderExt` PDA migration design — notes saved for the next session).
- The Squads multisig transfer of program upgrade authority (operational task, not a tech update).
- The Blowfish / Phantom false-positive removal submission (external process, not a code change).
- Audit firm outreach (OtterSec, Trail of Bits, Halborn) — ongoing, not in this scope.

Those are the five remaining items on the internal tracker as of this document's publication.

Prepared for internal review. All effort estimates are best-guess for a single experienced Solana/TypeScript engineer with full context on the existing codebase. Real-world timing will vary based on QA cycles, dependency surprises, and unplanned scope expansion. Treat these numbers as lower bounds, not contractual commitments.